

Greedy Algorithm

① Optimal solⁿ (max, min)
↳ Sorting ✓

② No fixed rule

③ lots of time global optimum solⁿ is not achieved.

Greedy Algo is the problem solving technique where we make the locally optimal choices ^(best) at each stage and try to achieve global optimum solⁿ.

Question:

Min Absolute difference pair:

Given 2 array A and B of same size. pair $(i, j) \mid i \in A, j \in A$

Sum = $\sum |i - j|$, find min. sum.

A = [1, 2, 3]

B = [2, 1, 3]

↳ combinations

Can 1 (1, 2) (2, 1) (3, 3) → Sum = 1 + 1 + 0 = 2

Can 2 (1, 1) (2, 2) (3, 3) → Sum = 0 + 0 + 0 = 0 (min)

Can 3 (1, 3) (2, 1) (3, 1) → Sum = 2 + 1 + 1 = 4

⋮

Sum = 0

for (i = 0 to n) {

Sum += Math.abs(A[i] - B[i]);

}

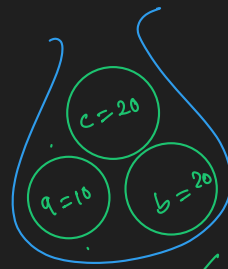
ans Sum

A → Sort [1, 2, 3]

B → Sort [1, 2, 3]

$$P/w = \begin{bmatrix} 15 & 25 & 30 \\ 6 & 5 & 4 \\ a & b & c \end{bmatrix}$$
 desc order

Product	a	b	c
P/w	6	5	4



$W = 50$

$$\text{Profit} = 6 \times 10 = 60$$

$$+ 5 \times 20 = 100$$

$$+ 4 \times 20 = 80$$

$Wt[b] \leq W?$

if true then take complete wt

$Wt[c] \leq W?$

no, then take the remain W of knapsack for the item 'c'

$$Wt = [30 \ 10 \ 20]$$

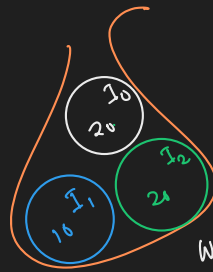
$$Price = [120 \ 60 \ 100]$$

map

item	wt	P/w
0	30	4
1	10	6
2	20	5

sort based on P/w in desc

item	wt	P/w
1	10	6
2	20	5
0	30	4



$W = 50$

$$\text{Profit} = 6 \times 10 + 5 \times 20 + 4 \times 20$$

$$= 60 + 100 + 80$$

$$= 240$$

Question 4

Activity Selection

$A_0 \ A_1 \ A_2 \ A_3 \ A_4 \ A_5$

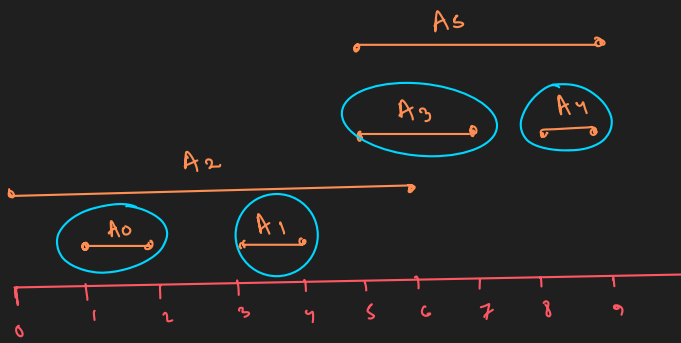
Start time = [1, 3, 0, 5, 8, 5]

$W_{(sort)}$ end time = [2, 4, 6, 7, 9, 9]

$A_{max} = 4$

Given all activities are sorted according to the end time

δ_{max} no. of max active perform w/o overlapping and print the activity.



non-overlapping activity

start time γ , end time of previous activity

input above sorting is not there based on end time.

$$s.t = [0, 1, 3, 5, 5, 8]$$

$$e.t = [6, 2, 4, 7, 9, 9]$$

$$Ans = A_1, A_2, A_3, A_4$$

Question 5

for each pair, (a, b) $a < b$ and you have to form a chain from those given pair st. $(a, b) (c, d)$ iff $b < c$ ^{largest}

$$pairs = \checkmark (5, 24)$$

$$(30, 60)$$

$$(5, 28)$$

$$\checkmark (27, 40)$$

$$\checkmark (50, 90)$$

pair 2 included

$$curr\ st > = \text{prev end value}$$

Job Sequencing Problem

job $\rightarrow$ (deadline, profit)

$$J_0 \quad 2, 80 \checkmark$$

$$J_1 \quad 6, 65 \checkmark$$

$$J_2 \quad 6, 70 \checkmark$$

$$J_3 \quad 5, 60 \checkmark$$

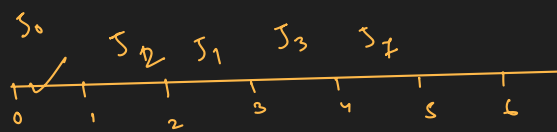
$$J_4 \quad 2, 22 \times$$

$$J_5 \quad 9, 25 \times$$

$$J_6 \quad 2, 10 \times$$

$$J_7 \quad 9, 20 \checkmark$$

max. profit $\uparrow$, at a time one job.
 $\hookrightarrow$ each job takes 1 unit of time



$$80 + 70 + 65 + 60 + 20$$

$$= 295$$

Ans 2
322

Out of Scope

Order

job1
(profit)

job2

↑ve

job1.profit - job2.profit

(asc order)

desc

job1

job2

job2

job1

↑ve

job1.profit - job2.profit

asc

job2 (2, 80)

job1 (6, 70)

J1 (6, 70)

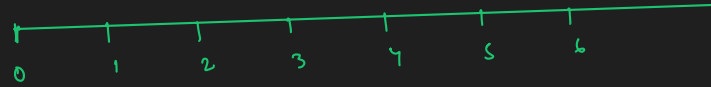
J2 (2, 80)

2 < 3 < 4

4 7 3 2

Comparison → result
 +ve → order maintain
 -ve → order doesn't maintain

	deadline	profit
J0	2	80
J1	6	65
J2	6	70
J3	5	60
J4	2	22
J5	4	25
J6	2	10
J7	4	20



Sort (profit)

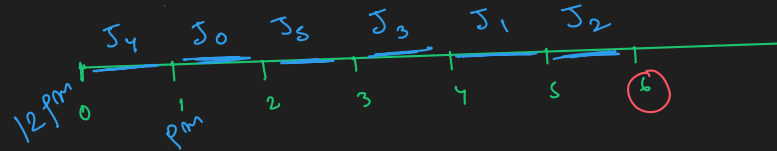
H.W
 f.c analysis

After Sorting

J0	2	80
J2	6	70
J1	6	65
J3	5	60
J5	4	25
J4	2	22
J7	4	20
J6	2	10

Profit = 322

max deadline = 6



Array [max deadline + 1] =

-1	4	0	5	3	1	2
0	1	2	3	4	5	6

```

Collections.sort(jobList,(a,b)-> b.profit-a.profit); // desc

int maxDeadline = 0;
for(int i=0; i<arr.length; i++){
    maxDeadline = Math.max(maxDeadline, arr[i][0]);
}

int jobSqn[] = new int[maxDeadline+1];
Arrays.fill(jobSqn, -1);

int maxProfit = 0;
for(int i=0; i<jobList.size(); i++){
    Job job = jobList.get(i); // most profitable job
    int idx = job.idx;
    int dl = job.deadline;
    int profit = job.profit;
    for(int j=dl; j>=1; j--){
        if(jobSqn[j] == -1){
            jobSqn[j] = idx;
            maxProfit += profit;
            break;
        }
    }
}

for(int i=0; i<jobSqn.length; i++){
    System.out.print(jobSqn[i]+" ");
}

System.out.println("\nMax. profit "+maxProfit);

```

CHOCOLA Problem

We are given a bar of chocolate composed of $m \times n$ square pieces. One should break the chocolate into single squares. Parts of the chocolate may be broken along the vertical and horizontal lines as indicated by the broken lines in the picture.

A single break of a part of the chocolate along a chosen vertical or horizontal line divides that part into two smaller ones. Each break of a part of the chocolate is charged a cost expressed by a positive integer. This cost does not depend on the size of the part that is being broken but only depends on the line the break goes along. Let us denote the costs of breaking along consecutive vertical lines with $x_1, x_2 \dots x_{m-1}$ and along horizontal lines with $y_1, y_2 \dots y_{n-1}$.

The cost of breaking the whole bar into single squares is the sum of the successive breaks. One should compute the minimal cost of breaking the whole chocolate into single squares.

